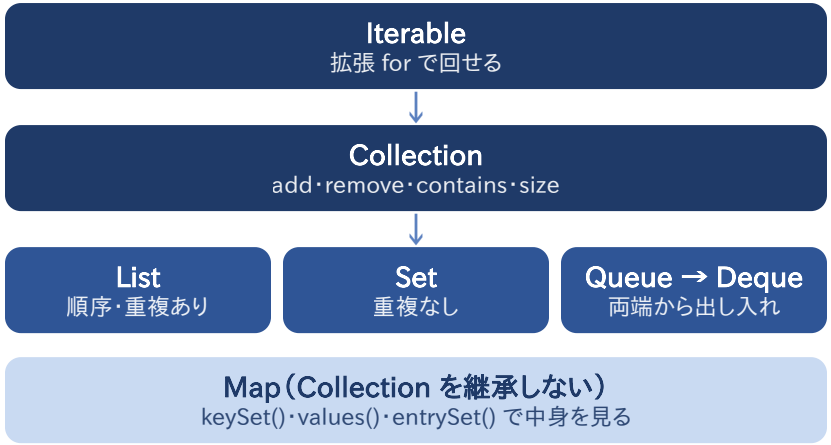
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 1 コレクションとジェネリクス　#01

コレクション・フレームワーク

List・Set・Map・Deque と、オートボクシングの落とし穴

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java 言語仕様 JLS、Java SE 17 の API ドキュメント）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 java.util のインタフェース



インタフェース	主な実装
List	ArrayList・LinkedList
Set	HashSet・LinkedHashSet・TreeSet
Deque	ArrayDeque・LinkedList
Map	HashMap・LinkedHashMap・TreeMap

`Collection<String> c = new HashMap<>();` はコンパイルエラー。

変数はインタフェース型、new のところだけ実装クラス。

出典：Oracle 試験内容チェックリスト 1／Java SE 17 API java.util.Collection・Map／The Java Tutorials: Collections

第1章

オートボクシング

int と Integer は自動で行き来するが、null と == に注意。

ボクシングとアンボクシング

int 10 ボクシング → Integer

Integer アンボクシング → int

```
Integer a = 127, b = 127;  // a == b → true
Integer c = 128, d = 128;  // c == d → false
c.equals(d)                // true
```

-128〜127 は同じオブジェクト（JLS 5.1.7）。値は equals で比べる。

コード	結果
`Long l = 5;`	コンパイルエラー
`Integer i = 5; long l = i;`	OK（5）
`int n = map.get("無いキー");`	NullPointerException
`i.equals(10L)`	false（型が違う）
`l == 10`（Long l = 10L）	true（数で比較）

ボクシングは対応する型だけ。int → Long はできない。

出典：JLS 5.1.7 Boxing Conversion・5.1.8 Unboxing Conversion・5.2 Assignment Contexts／JDK 17 での実行結果

remove(int) と remove(Object)

```
List<Integer> list =
  new ArrayList<>(List.of(10, 20, 30, 1));
list.remove(1);                  // 添字1の20を消す
list.remove(Integer.valueOf(1)); // 値1を消す
System.out.println(list);        // [10, 30]
```

オーバーロードの解決の順（JLS 15.12.2）

1. ボクシングも可変長引数も使わずに合う
2. ボクシングを使って合う
3. 可変長引数を使って合う

> remove(1) は段階1で remove(int) が見つかるので、remove(Object) は候補にならない。

出典：JLS 15.12.2 Compile-Time Step 2／API List.remove(int)・remove(Object)／JDK 17 での実行結果

第2章

List と Set

順番を持つか、重複を許すか、並べ替えるか。

List と equals

	ArrayList	LinkedList
しくみ	配列	前後の参照でつなぐ
添字で取得	速い	先頭からたどる
先頭の出し入れ	ずらすので遅い	速い（Deque も実装）
null	入る	入る

```
a: ArrayList  [1, 2]
b: LinkedList [1, 2]
c: HashSet    [1, 2]
d: TreeSet    [1, 2]
a.equals(b) // true  同じ順・同じ要素の List
a.equals(c) // false List と Set
c.equals(d) // true  同じ要素の Set
```

> 型引数は参照型だけ。`List<int>` はエラー、`List<int[]>` は OK。

出典：API List.equals・Set.equals・ArrayList・LinkedList／JLS 4.5.1／JDK 17 での実行結果

Set の三つの実装

実装	並び順	同じかの判断	null
HashSet	決まらない	equals と hashCode	1 つ入る
LinkedHashSet	入れた順	equals と hashCode	1 つ入る
TreeSet	自然順序か Comparator の順	compareTo が 0	自然順序では NPE

```
class Item { String name; }   // equals なし
record Tag(String name) {}    // 自動で作られる
HashSet に Item("a") を2回 → size 2
HashSet に Tag("a") を2回  → size 1
TreeSet に Item を add     → ClassCastException
```

文字列の自然順序（文字コード順）

[10, 9, A, B, a, b]

数字 → 大文字 → 小文字。"10" は '1' で比べて "9" より前

出典：API HashSet・LinkedHashSet・TreeSet・Object.hashCode／JLS 8.10.3／JDK 17 での実行結果

並んだ Set と Map を探す

{10, 20, 30, 40} で

メソッド	意味	例
lower(x)	x 未満の最大	lower(10) → null
floor(x)	x 以下の最大	floor(25) → 20
ceiling(x)	x 以上の最小	ceiling(25) → 30
higher(x)	x より大きい最小	higher(30) → 40

> TreeMap は floorKey・ceilingKey など。

{1, 3, 5, 7, 9} の部分のビュー

subSet(3, 7)	[3, 5]（始まり含む・終わり含まない）
headSet(5)	[1, 3]
headSet(5, true)	[1, 3, 5]
tailSet(5)	[5, 7, 9]
descendingSet().first()	9

出典：API NavigableSet・NavigableMap／JDK 17 での実行結果

第3章

Map

キーと値の組。戻り値は「前の値」。

Map の実装と null

実装	順番	null キー
HashMap	決まらない	1 つ OK
LinkedHashMap	入れた順	1 つ OK
TreeMap	キーの順	NPE（値は OK）
Map.of	決まらない	NPE・重複キーは IAE

LinkedHashMap に同じキーを put し直しても順番は変わらない。

ビュー（元の Map とつながっている）

keySet()
remove するとエントリも消える

values()
値の Collection

entrySet()
Map.Entry の setValue で値を書き換え

get が null → 「キーがない」か「値が null」か。containsKey で区別。

出典：API HashMap・LinkedHashMap・TreeMap・Map.of・Map.keySet・Map.Entry／JDK 17 での実行結果

Map の便利なメソッド

メソッド	キーがない	キーがある
put	入れて null	上書きして前の値
putIfAbsent	入れて null	何もせず今の値
getOrDefault	既定値	値
merge(k, v, f)	v を入れる	f(前, v)。null なら削除
computeIfAbsent	f(k) を入れて返す	今の値を返す

```
// 数え上げ
count.merge(w, 1, Integer::sum);
// "b a b c b a" → {a=2, b=3, c=1}

// グループ分け
byLen.computeIfAbsent(w.length(),
    k -> new ArrayList<>()).add(w);
// {3=[ant, bee, cat], 4=[deer]}
```

出典：API Map.put・putIfAbsent・getOrDefault・merge・computeIfAbsent／JDK 17 での実行結果

第4章

Queue と Deque

例外を投げる版と、null を返す版がある。

Queue と Deque のメソッド

操作	失敗で例外	失敗で特別な値
入れる	add	offer → false
取り出す	remove	poll → null
見る	element	peek → null

空のとき remove・element・pop は NoSuchElementException。

ArrayDeque は null を入れられない（offer でも NPE）。LinkedList は入る。

スタックとして（先頭を使う）

```
d.push(1); d.push(2); d.push(3); // [3, 2, 1]
d.pop()      // 3
d.peek()     // 2 (peekFirst)
d.peekLast() // 1
```

キューとして（末尾に入れ先頭から出す）

```
q.offer("A"); q.offer("B"); q.add("C");
q.poll()          // "A"
d.offerFirst("Z") // [Z, B, C]
```

出典：API Queue・Deque（メソッドの要約表）・ArrayDeque／JDK 17 での実行結果

第5章

イテレータと変更できないコレクション

回しながら消すときと、変えられない List の見分け方。

回しながら消す

```
for (String s : list) {          // [a, b, c]
  if (s.equals("a")) list.remove(s);
}  // → ConcurrentModificationException

for (String s : list) {
  if (s.equals("b")) list.remove(s);
}  // → [a, c]（例外なし）
```

最後から2番目を消すと hasNext が false になり、next が呼ばれない。フェイルファストは保証ではない。

安全な消し方

```
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
  if (it.next() % 2 == 0) it.remove();
}
list.removeIf(n -> n > 4);
```

[1, 2, 3, 4, 5] → 偶数を消して [1, 3, 5] → 4 より大きいものを消して [1, 3]。

出典：API Iterator・ConcurrentModificationException・Collection.removeIf／JDK 17 での実行結果

変更できないコレクション

作り方	set	add・remove	null	元の変更
List.of(...)	UOE	UOE	NPE	—
Arrays.asList(...)	○	UOE	○	配列と共有
List.copyOf(c)	UOE	UOE	NPE	見えない（コピー）
Collections.unmodifiableList(c)	UOE	UOE	○	見える（ビュー）

Set.of・Map.of に重複を渡すと IllegalArgumentException（duplicate element／duplicate key）。

UOE＝UnsupportedOperationException、NPE＝NullPointerException。

出典：API List.of（Unmodifiable Lists）・Arrays.asList・List.copyOf・Collections.unmodifiableList・Set.of／JDK 17 での実行結果